

ASSIGNMENT-11.2

NAME: SRAVANI ANUMALA

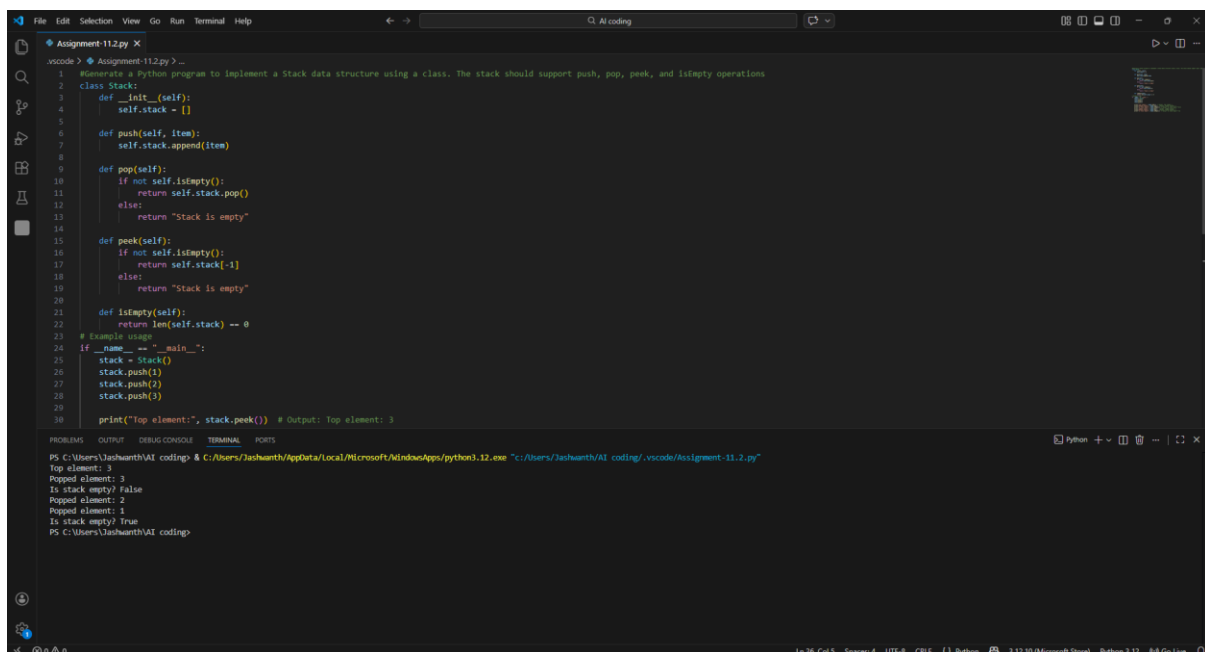
ROLLNO: 2303A510G7

BATCH: 30

TASK-1

PROMPT:Generate a Python program to implement a Stack data structure using a class. The stack should support push, pop, peek, and isEmpty operations

CODE:



```
1 #Generate a Python program to implement a Stack data structure using a class. The stack should support push, pop, peek, and isEmpty operations
2 class Stack:
3     def __init__(self):
4         self.stack = []
5
6     def push(self, item):
7         self.stack.append(item)
8
9     def pop(self):
10        if not self.isEmpty():
11            return self.stack.pop()
12        else:
13            return "Stack is empty"
14
15    def peek(self):
16        if not self.isEmpty():
17            return self.stack[-1]
18        else:
19            return "Stack is empty"
20
21    def isEmpty(self):
22        return len(self.stack) == 0
23
24 # Example usage
25 if __name__ == "__main__":
26     stack = Stack()
27     stack.push(1)
28     stack.push(2)
29     stack.push(3)
30     print("Top element:", stack.peek()) # Output: Top element: 3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Jashwanth\AI coding> C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\Jashwanth\AI coding\vscode\Assignment-11.2.py"
Top element: 3
Popped element: 3
Is stack empty? False
Popped element: 2
Popped element: 1
Is stack empty? True
PS C:\Users\Jashwanth\AI coding>
```

OBSERVATION:

A **Stack** is a linear data structure that follows **LIFO (Last In First Out)** principle.

This means:

- The **last element inserted** is the **first element removed**.

Operations

1. **Push** – Add element to the stack
2. **Pop** – Remove top element
3. **Peek** – View top element
4. **Empty** – Check whether stack is empty

Real Life Example

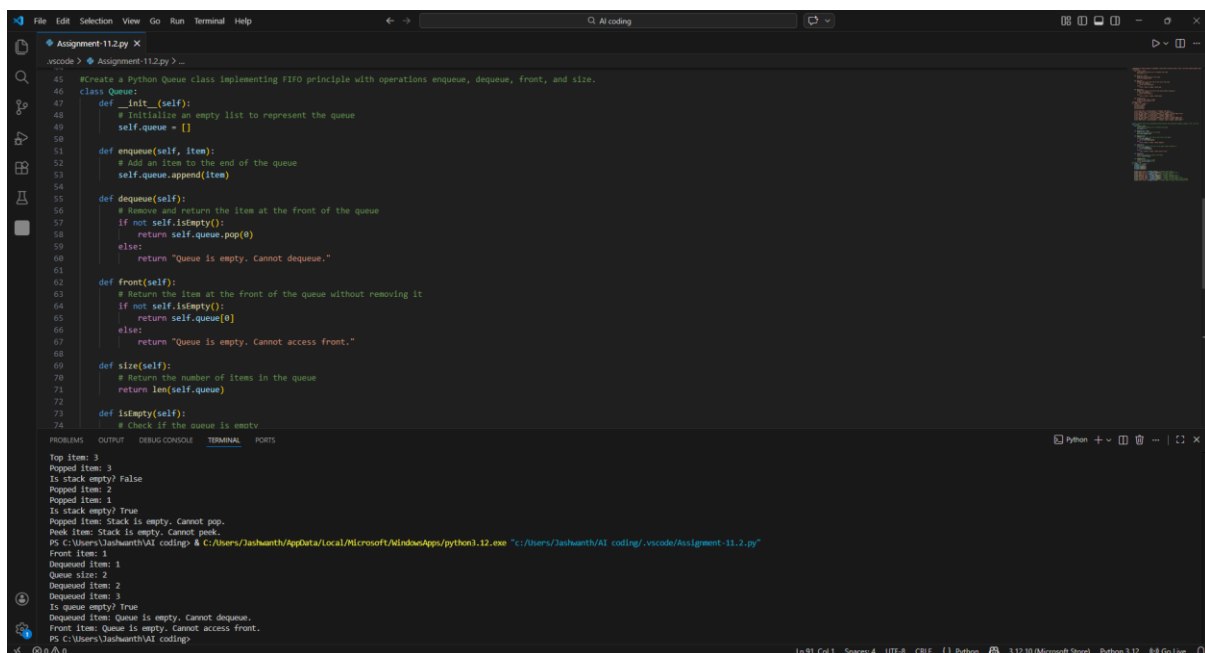
- Stack of plates
- Browser back button

A stack was built that works like a pile — last in, first out. You can add items (push), remove them (pop), look at the top (peek), and check if it's empty. The output showed items leaving in the reverse order they were added, following the LIFO rule.

TASK-2

PROMPT: Create a Python Queue class implementing FIFO principle with operations enqueue, dequeue, front, and size.

CODE:



```

45 #Create a Python Queue class implementing FIFO principle with operations enqueue, dequeue, front, and size.
46 class Queue:
47     def __init__(self):
48         # Initialize an empty list to represent the queue
49         self.queue = []
50
51     def enqueue(self, item):
52         # Add an item to the end of the queue
53         self.queue.append(item)
54
55     def dequeue(self):
56         # Remove and return the item at the front of the queue
57         if not self.isempty():
58             return self.queue.pop(0)
59         else:
60             return "Queue is empty. Cannot dequeue."
61
62     def front(self):
63         # Return the item at the front of the queue without removing it
64         if not self.isempty():
65             return self.queue[0]
66         else:
67             return "Queue is empty. Cannot access front."
68
69     def size(self):
70         # Return the number of items in the queue
71         return len(self.queue)
72
73     def isempty(self):
74         # Check if the queue is empty

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Top Item: 3
Popped Item: 3
Is stack empty? False
Popped Item: 2
Popped Item: 1
Is stack empty? True
Popped Item: Stack is empty. Cannot pop.
Peek Item: Stack is empty. Cannot peek.
PS C:\Users\Jashwanth\AI coding> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\Jashwanth\AI coding\vscode\Assignment-11.2.py"
Front Item: 1
Dequeued Item: 1
Queue size: 2
Dequeued Item: 2
Dequeued Item: 3
Is queue empty? True
Dequeued Item: Queue is empty. Cannot dequeue.
Front Item: Queue is empty. Cannot access front.
PS C:\Users\Jashwanth\AI coding>

```

OBSERVATION:

A Queue follows **FIFO (First In First Out)**.

Meaning:

- The **first element added** will be **first removed**.

Operations

1. **Enqueue** – Add element to queue
2. **Dequeue** – Remove element
3. **Front** – View first element
4. **Size** – Count elements

Real Life Example

- People waiting in line
- Printer queue

A queue was built that works like a line — first in, first out. You can add items (enqueue), remove them (dequeue), check the front, see the size, and test if it's empty. The output showed items leaving in the same order they entered.

TASK-3

PROMPT: Write a Python program implementing a singly linked list with node creation, insertion at end, and traversal operations.

CODE:

```

93 #Write a Python program implementing a singly linked list with node creation, insertion at end, and traversal operations.
94 class Node:
95     def __init__(self, data):
96         # Initialize a node with data and a pointer to the next node
97         self.data = data
98         self.next = None
99 class SinglyLinkedList:
100     def __init__(self):
101         # Initialize an empty linked list with a head pointer
102         self.head = None
103
104     def insert_at_end(self, data):
105         # Create a new node with the given data
106         new_node = Node(data)
107         # If the linked list is empty, set the new node as the head
108         if self.head is None:
109             self.head = new_node
110             return
111         # Otherwise, traverse to the end of the list and add the new node
112         last_node = self.head
113         while last_node.next:
114             last_node = last_node.next
115         last_node.next = new_node
116
117     def traverse(self):
118         # Traverse the linked list and print each node's data
119         current_node = self.head
120         while current_node:
121             print(current_node.data)
122             current_node = current_node.next

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Popped item: Stack is empty. Cannot pop.
Push Item: Stack is empty. Cannot push.
Front Item: 1
Dequeued Item: 1
Queue size: 2
Dequeued Item: 2
Dequeued Item: 3
Is queue empty? True
Dequeued Item: Queue is empty. Cannot dequeue.
Front Item: Queue is empty. Cannot access front.
PS C:\Users\Jashwanth\AI coding>
PS C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/Jashwanth/ai coding/.vscode/Assignment-11.2.py"
> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/Jashwanth/ai coding/.vscode/Assignment-11.2.py"
Linked List Traversal:
1
2
3
PS C:\Users\Jashwanth\AI coding>

```

OBSERVATION:

A **Linked List** is a dynamic data structure where each element is called a **node**.

Each node contains:

1. **Data**
2. **Next pointer** (address of next node)

Operations

- Node creation
- Insertion
- Traversal

Advantage

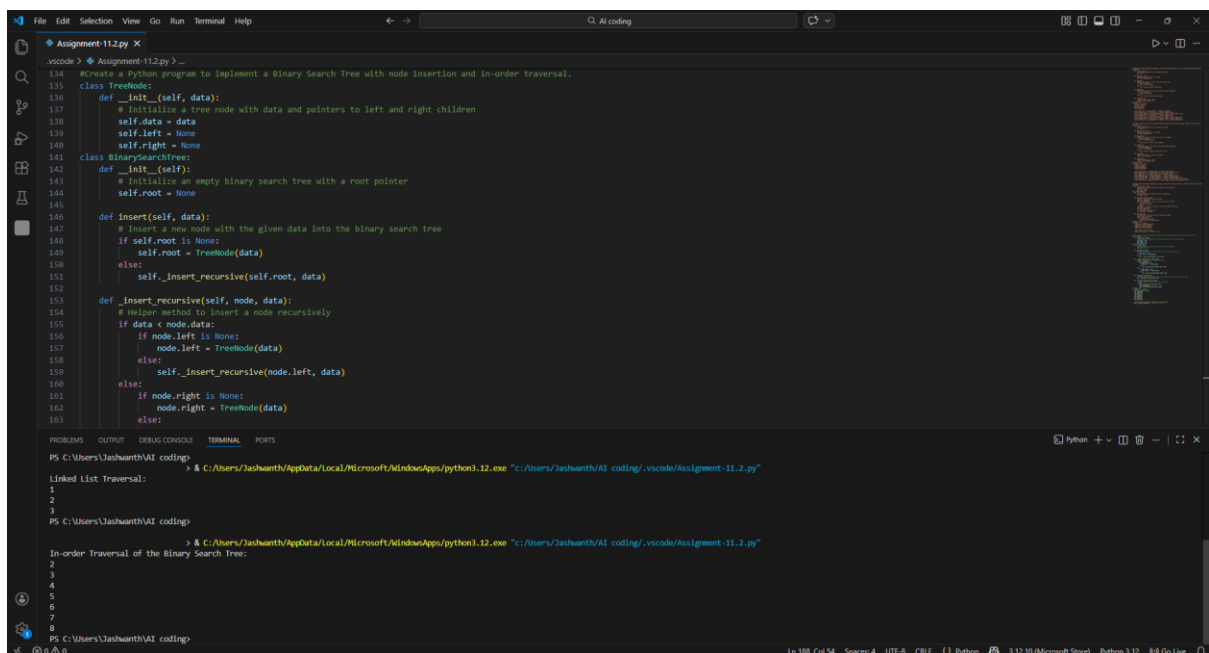
- Dynamic size
- Efficient insertion

A linked list was created where each node points to the next. You can insert nodes at the end and traverse to display all items. The output confirmed elements were stored and shown in order.

TASK-4

PROMPT: Create a Python program to implement a Binary Search Tree with node insertion and in-order traversal.

CODE:



```

134 #Create a Python program to implement a Binary Search Tree with node insertion and in-order traversal.
135 class TreeNod:
136     def __init__(self, data):
137         # Initialize a tree node with data and pointers to left and right children
138         self.data = data
139         self.left = None
140         self.right = None
141 class BinarySearchTree:
142     def __init__(self):
143         # Initialize an empty binary search tree with a root pointer
144         self.root = None
145
146     def insert(self, data):
147         # Insert a new node with the given data into the binary search tree
148         if self.root is None:
149             self.root = TreeNod(data)
150         else:
151             self._insert_recursive(self.root, data)
152
153     def _insert_recursive(self, node, data):
154         # Helper method to insert a node recursively
155         if data < node.data:
156             if node.left is None:
157                 node.left = TreeNod(data)
158             else:
159                 self._insert_recursive(node.left, data)
160         else:
161             if node.right is None:
162                 node.right = TreeNod(data)
163             else:
164                 self._insert_recursive(node.right, data)
165
166     def in_order_traversal(self):
167         # Perform in-order traversal of the binary search tree
168         self._in_order_recursive(self.root)
169
170     def _in_order_recursive(self, node):
171         # Helper method for in-order traversal
172         if node:
173             self._in_order_recursive(node.left)
174             print(node.data, end=" ")
175             self._in_order_recursive(node.right)
176
177 def main():
178     bst = BinarySearchTree()
179     bst.insert(5)
180     bst.insert(3)
181     bst.insert(7)
182     bst.insert(2)
183     bst.insert(4)
184     bst.in_order_traversal()
185
186 if __name__ == "__main__":
187     main()

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Jashwanth\AI coding>
> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\Jashwanth\AI coding\vscode\Assignment-11.2.py"
Linked List Traversal:
1
2
3
PS C:\Users\Jashwanth\AI coding>
> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\Jashwanth\AI coding\vscode\Assignment-11.2.py"
In-order Traversal of the Binary Search Tree:
2
3
4
5
6
7
8
PS C:\Users\Jashwanth\AI coding>

```

OBSERVATION:

A **Binary Search Tree (BST)** is a hierarchical structure where:

- Left child < Parent
- Right child > Parent

Operations

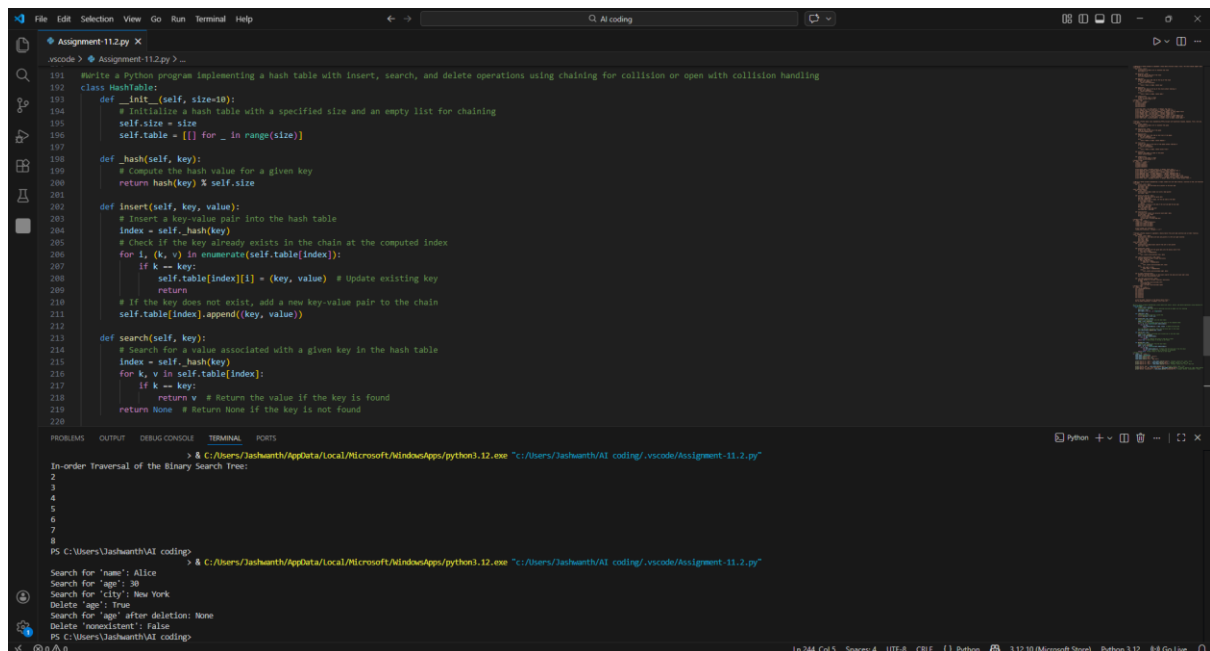
1. Insert node
2. In-order traversal

A tree structure was made where smaller values go left and larger go right. Insertions placed nodes correctly, and in-order traversal printed elements in ascending order.

TASK-5

PROMPT: Write a Python program implementing a hash table with insert, search, and delete operations using chaining for collision or open with collision handling.

CODE:



```
191 Write a Python program implementing a hash table with insert, search, and delete operations using chaining for collision or open with collision handling
192 class HashTable:
193     def __init__(self, size=10):
194         # Initialize a hash table with a specified size and an empty list for chaining
195         self.size = size
196         self.table = [[] for _ in range(size)]
197
198     def _hash(self, key):
199         # Compute the hash value for a given key
200         return hash(key) % self.size
201
202     def insert(self, key, value):
203         # Insert a key-value pair into the hash table
204         index = self._hash(key)
205         # Check if the key already exists in the chain at the computed index
206         for i, (k, v) in enumerate(self.table[index]):
207             if k == key:
208                 self.table[index][i] = (key, value) # Update existing key
209                 return
210         # If the key does not exist, add a new key-value pair to the chain
211         self.table[index].append((key, value))
212
213     def search(self, key):
214         # Search for a value associated with a given key in the hash table
215         index = self._hash(key)
216         for k, v in self.table[index]:
217             if k == key:
218                 return v # Return the value if the key is found
219         return None # Return None if the key is not found
220
221
222
> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\Jashwanth\AI coding\.vscode\Assignment-11.2.py"
In-order Traversal of the Binary Search Tree:
2
3
4
5
6
7
8
PS C:\Users\Jashwanth\AI coding>
Search for 'name': Alice
Search for 'age': 30
Search for 'city': New York
Delete 'age': True
Search for 'age' after deletion: None
Delete 'nonexistent': False
PS C:\Users\Jashwanth\AI coding>
```

OBSERVATION:

A **Hash Table** stores data using a **hash function**.

Hash function converts key into **index position**.

Problem

Two keys may map to same index → **Collision**

Solution

Collision handling methods:

1. **Chaining**
2. **Open Addressing**

Here we use **chaining**.

A hash table was implemented using a hash function and chaining for collisions. It supported insert, search, and delete. The output showed fast storage and retrieval, with keys handled correctly.